



TECNOLÓGICO
NACIONAL DE MÉXICO



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



INSTITUTO TECNOLÓGICO DE OAXACA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES

NOMBRE DEL PROYECTO:

Desarrollar una aplicación web para la administración
de los procesos para la Oficina de proyectos de
Vinculación del Departamento de Sistemas y
Computación

PRESENTAN:

Hernández Ramírez Osiris - 19161231

Oaxaca de Juárez, Oaxaca. junio de 2025

Contenido

CAPÍTULO I. MARCO CONTEXTUAL	4
1.1 GENERALIDADES DE LA INSTITUCIÓN	5
1.1.1 Nombre de la Institución	5
1.1.2 Giro de la Institución	5
1.1.3 Dirección (Croquis de ubicación)	5
1.1.4 Organigrama general de la Institución.	6
1.1.5 Organigrama específico del área.	7
1.1.6 Breve descripción de los procesos de la Institución	7
1.2 GENERALIDADES EL PROYECTO	8
1.2.1 Nombre del proyecto	8
1.2.2 Planteamiento del problema	8
1.2.3 Objetivos	9
1.2.3.1 Objetivo general	9
1.2.3.2 Objetivos Específicos	9
1.2.4 Justificación	9
1.2.5 Alcances y limitaciones	9
1.2.5.1 Alcances	9
1.2.5.2 Limitaciones	10
CAPÍTULO II. MARCO TEÓRICO	11
2.1 METODOLOGÍA	12
2.1.1 Metodologías de desarrollo tradicional	12
2.1.1.1 Definición	12
2.1.1.2 Características	12
2.1.1.3 Tipos	12
2.1.1.4 Cuadro comparativo	12
2.1.2 Metodología a usar: Metodología Incremental	13
2.2 FRAMEWORKS	14
2.2.1 Front-end	14
2.2.1.1 Definición	14
2.2.1.2 Características	14
2.2.1.3 Cuadro comparativo	14
2.2.1.4 Framework de desarrollo web a utilizar: Angular	14
2.2.2 Back-end	15
2.2.2.1 Definición	15
2.2.2.2 Características	15
2.2.2.3 Cuadro comparativo	15
2.2.2.4 Framework de desarrollo de servicios web a utilizar: ASP.NET	15
2.3 SISTEMA GESTOR DE BASE DE DATOS	16
2.3.1 Definición	16
2.3.2 Funciones	16
2.3.3 Cuadro comparativo de Sistemas Gestores de Base de Datos	16
2.3.4 Sistema Gestor de Base de Datos a utilizar: MySQL	17
CAPITULO III. DESARROLLO METODOLOGICO	17
3.1 INCREMENTO 0: ANÁLISIS Y PLANEACIÓN DEL PROYECTO	18
3.1.1 Análisis de Requerimientos	18
3.1.2 Análisis del entorno tecnológico	18
3.1.3 Diseño general del sistema	19

3.1.4 Planificación del Proyecto	22
3.1.5 Configuración del entorno de desarrollo	25
3.2 INCREMENTO 1: LOGIN Y MÓDULOS DE REGISTRO (ESTUDIANTES, DOCENTES, PROYECTOS Y EMPRESAS).	25
3.2.1 Planificación y Análisis de Requisitos	25
3.2.1.1 Casos de Uso	26
3.2.1.2 Historias de Usuario	26
3.2.2 Diseño y Desarrollo	31
3.2.2.1 Diagrama Entidad-Relación	31
3.2.2.3 Diagrama Entidad-Relación para la funcionalidad del sistema	31
3.2.2.4 Diagrama Relacional	31
3.2.2.5 Diagrama Relacional para el funcionamiento del sistema	32
3.2.2.6 Diccionario de Datos	32
3.2.3 Pruebas e Integración	46
3.2.3.1 Login	46
3.2.3.2 Estudiantes	46
3.2.3.3 Docentes	46
3.2.3.4 Empresas	46
3.2.3.5 Proyectos	46
3.2.4 Entrega y Retroalimentación	46
3.3 INCREMENTO 2: BANCO DE PROYECTOS Y ANTEPROYECTO	46
3.3.1 Planificación y Análisis de Requisitos	46
3.3.2 Diseño y Planificación	46
3.3.3 Pruebas e Integración	47
3.3.4 Entrega y Retroalimentación	47
3.4 INCREMENTO 3: ASIGNACIÓN DE ASESORES Y REVISORES	47
3.4.1 Planificación y Análisis de Requisitos	47
3.4.2 Diseño y Planificación	47
3.4.3 Pruebas e Integración	47
3.4.4 Entrega y Retroalimentación	47
3.5 INCREMENTO 4: EXPEDIENTES DE PROYECTOS DE RESIDENCIA	47
3.5.1 Planificación y Análisis de Requisitos	47
3.5.2 Diseño y Planificación	47
3.5.3 Pruebas e Integración	47
3.5.4 Entrega y Retroalimentación	47
3.6 CIERRE DEL PROYECTO	47
REFERENCIAS	47

CAPÍTULO I. MARCO CONTEXTUAL

1.1 GENERALIDADES DE LA INSTITUCIÓN

1.1.1 Nombre de la Institución

Instituto Tecnológico de Oaxaca

1.1.2 Giro de la Institución

Educación superior tecnológica pública. Formación de profesionistas en diversas áreas de la ingeniería y tecnología, así como investigación, desarrollo e innovación.

1.1.3 Dirección (Croquis de ubicación)

Avenida Ing. Víctor Bravo Ahuja No. 125 Esquina Calzada Tecnológico, C.P. 68030

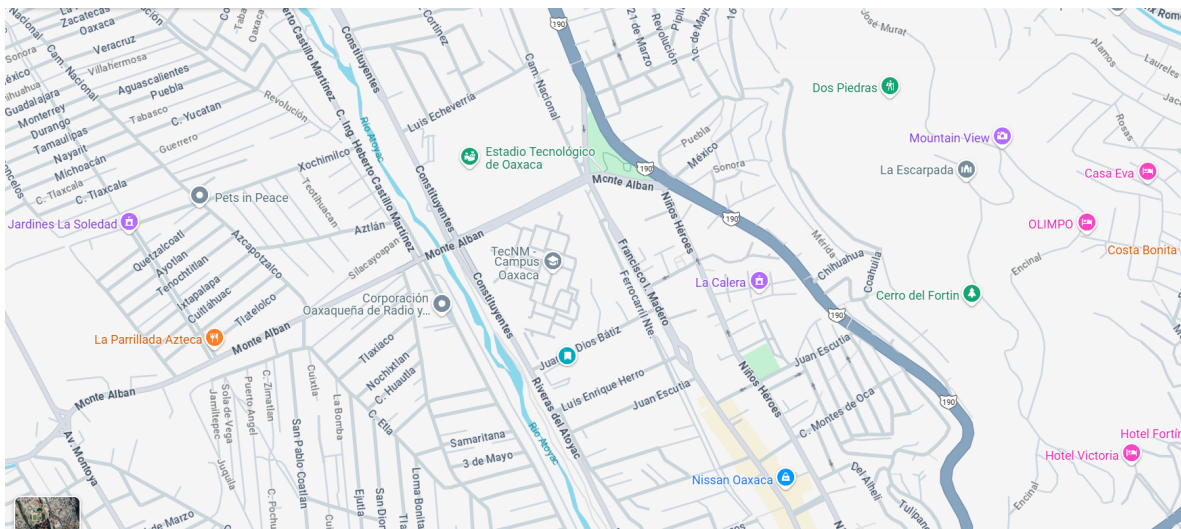
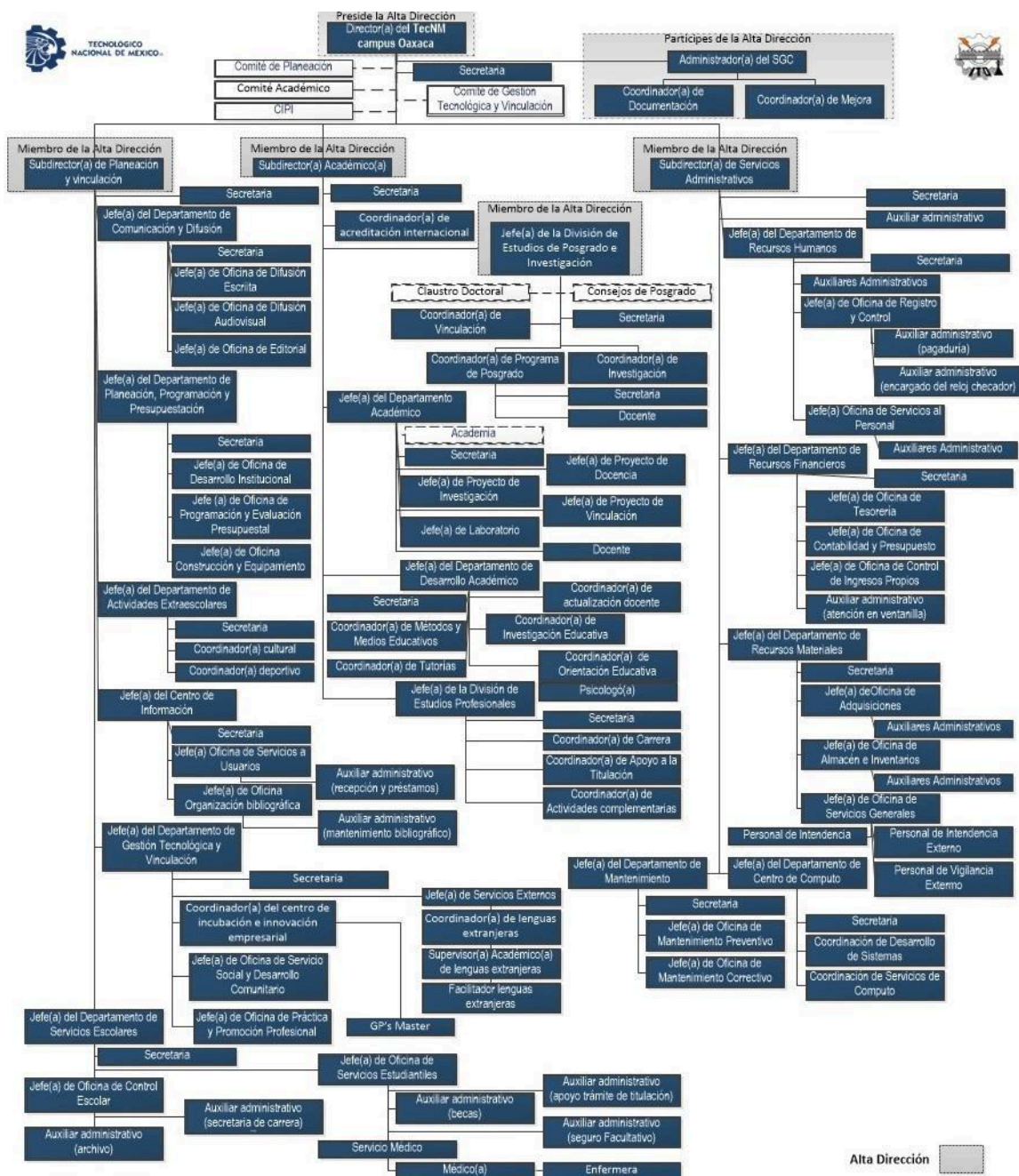
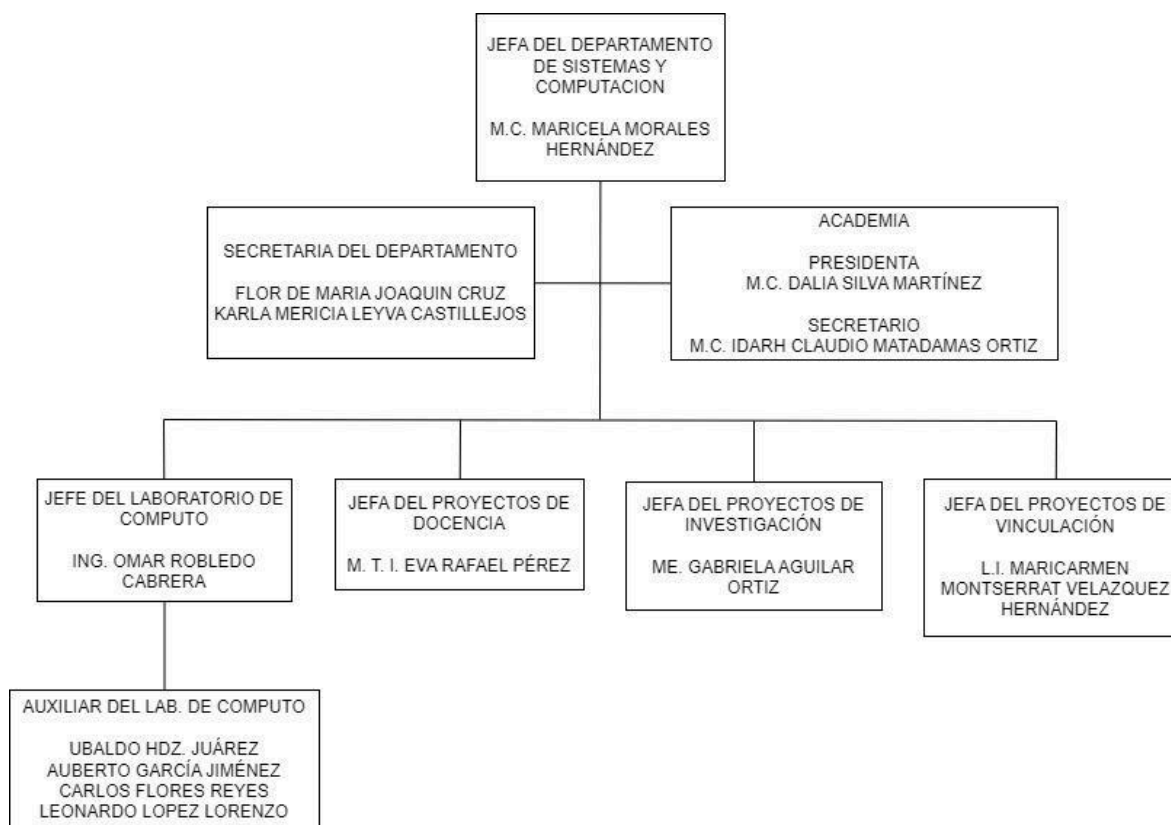


Figura 1. Croquis de ubicación

1.1.4 Organigrama general de la Institución.



1.1.5 Organigrama específico del área.



1.1.6 Breve descripción de los procesos de la Institución

El Instituto Tecnológico de Oaxaca (ITO) es una institución pública de educación superior que forma parte del Tecnológico Nacional de México. Su misión principal es la formación integral de profesionistas en diversas áreas de la ingeniería y la tecnología, así como la promoción de la investigación, el desarrollo tecnológico, la innovación y la vinculación con los sectores productivo, social y gubernamental.

La estructura académica y administrativa del ITO contempla diversos **departamentos académicos**, entre los que se encuentra el **Departamento de Sistemas y Computación**, responsable de coordinar programas educativos, actividades docentes y de vinculación con el entorno profesional.

Dentro de estos procesos institucionales, la **vinculación** juega un papel clave al establecer puentes entre el entorno académico y el sector productivo. A través de la **Oficina de Proyectos de Vinculación**, se coordinan y gestionan proyectos que permiten a los estudiantes aplicar sus conocimientos en escenarios reales, tales como residencias profesionales, proyectos integradores, estancias técnicas y colaboración con empresas o instituciones.

El **Jefe de Proyecto de Vinculación** es responsable de coordinar el diseño, desarrollo, seguimiento y evaluación de estos proyectos, en estrecha colaboración con el jefe de departamento, docentes, asesores y estudiantes. Entre sus funciones se encuentran la propuesta y planeación anual de proyectos, la integración de equipos de trabajo, el monitoreo de avances, la difusión de resultados y la evaluación final del impacto académico y social de los proyectos realizados.

En conjunto, estos procesos permiten al Instituto cumplir con su función formativa y contribuir activamente al desarrollo regional, fortaleciendo la preparación profesional de los estudiantes mediante experiencias prácticas alineadas con su perfil de egreso.

1.2 GENERALIDADES EL PROYECTO

1.2.1 Nombre del proyecto

Sistema web para la gestión integral de las residencias profesionales del Departamento de Sistemas y Computación del Instituto Tecnológico de Oaxaca.

1.2.2 Planteamiento del problema

En el Instituto Tecnológico de Oaxaca, el proceso de gestión de las residencias profesionales presenta deficiencias importantes en su organización, control y seguimiento. Actualmente, muchas de las actividades relacionadas con la vinculación, el registro de proyectos, la entrega de documentación, la evaluación de avances y el almacenamiento de evidencias, se realizará de forma manual o mediante herramientas no integradas, como correos electrónicos, hojas de calculo o carpetas físicas/digitales dispersas. Esta situación genera cuellos de botella administrativos, pérdida de información, dificultades para el seguimiento puntual del avance de los estudiantes y una baja eficiencia en la vinculación con empresas o proyectos internos.

Los estudiantes enfrentan incertidumbre sobre los plazos, los formatos requeridos y el estado de su proceso de residencia. Asimismo, no existe una plataforma institucional que permita gestionar de forma automatizada las fechas clave, hacer revisión y validación de documentos, ni llevar un control del expediente académico-administrativo del residente. Del lado académico, los asesores internos y personal administrativo no cuentan con una herramienta que permita dar seguimiento sistematizado al cumplimiento de actividades, carga de reportes, ni al historial de avances o evaluación del estudiante.

No hay existencia de un repositorio institucional de proyectos disponibles, lo que limita la posibilidad de los alumnos accedan fácilmente a propuestas de residencia generadas por el propio instituto o por empresas vinculadas. Esto representa una barrera para el aprovechamiento de oportunidades y una limitación para la planeación estratégica del desarrollo profesional del estudiante.

1.2.3 Objetivos

1.2.3.1 Objetivo general

Diseñar e implementar un sistema web integral para la gestión de las residencias profesionales en el Instituto Tecnológico de Oaxaca, en la carrera de Ingeniería en Sistemas Computacionales, que permita automatizar y controlar eficientemente el registro, vinculación, seguimiento, evaluación y documentación de los proyectos y estudiantes.

1.2.3.2 Objetivos Específicos

- Centralizar el registro de proyectos de residencia disponibles y asignación de estudiantes.
- Automatizar la entrega y revisión de documentos en cada etapa del proceso.
- Gestionar de forma eficiente los plazos, avances y reportes de cada estudiante.
- Facilitar la asignación y el seguimiento por parte de asesores internos.
- Crear un repositorio institucional de proyectos anteriores y disponibles.
- Mejorar la comunicación entre estudiantes, asesores y la oficina de vinculación.

1.2.4 Justificación

El sistema propuesto permitirá resolver la dispersión de información y la falta de control sobre los procesos de residencia profesional, mediante una plataforma centralizada y accesible vía web. Se utilizarán tecnologías como Angular, ASP.NET Core, y bases de datos relacionales como MySql para garantizar la escalabilidad y seguridad del sistema. Entre los beneficios destacan la eficiencia administrativa, la reducción del uso de papel y la posibilidad de hacer estadísticas para la mejora continua del proceso de vinculación. Al igual, centralizara todas las etapas del proceso, desde el registró y vinculación de proyectos, pasando por la entrega de documentación y reportes, la asignación y seguimiento de asesores, hasta la evaluación final y cierre del expediente del estudiante. El sistema también permitirá una gestión más estricta y automatizada de los tiempo y fechas limite, así como incorporar un repositorio de proyectos propuestos por empresas o departamentos académicos.

1.2.5 Alcances y limitaciones

1.2.5.1 Alcances

- Módulo de registro de estudiantes y proyectos.
- Módulo para la asignación de asesores y seguimiento de avances.
- Sistema de carga y revisión de documentos.
- Control de fechas clave y notificaciones automáticas.
- Repositorio digital de proyectos concluidos.
- Panel de administración para la oficina de vinculación.

1.2.5.2 Limitaciones

- Requiere conexión a Internet para su funcionamiento.

- Posibles resistencias al cambio del personal acostumbrado a procesos manuales.
- El sistema no contempla integración inmediata con otras plataformas institucionales (por ejemplo, el sistema de calificaciones).

CAPÍTULO II. MARCO TEÓRICO

2.1 METODOLOGÍA

2.1.1 Metodologías de desarrollo tradicional

2.1.1.1 Definición

Las metodologías de desarrollo tradicionales, también llamadas metodologías en cascada o secuenciales, son modelos de ingeniería de software que se caracterizan por seguir un proceso estructurado y ordenado. En ellas, las etapas del desarrollo (análisis, diseño, implementación, pruebas y mantenimiento) se ejecutan de manera lineal, donde cada fase debe completarse antes de iniciar la siguiente.

Según Sommerville (2011),

“Los procesos tradicionales de desarrollo de software siguen un flujo de actividades predefinido y rígido, en el cual los cambios posteriores a la fase de requisitos suelen ser costosos y difíciles de implementar.”

2.1.1.2 Características

- Estructura rígida y secuencial.
- Documentación exhaustiva en cada fase.
- Bajo nivel de flexibilidad frente a cambios.
- Enfoque predictivo (todo se planifica desde el inicio).
- Ideal para proyectos con requisitos muy claros y estables.

2.1.1.3 Tipos

Los modelos tradicionales más conocidos incluyen:

- Modelo en Cascada: el primero en proponerse, de flujo lineal.
- Modelo en V: integra validación y verificación simultáneamente.
- Modelo Espiral: combina el enfoque secuencial con iteraciones controladas por riesgo.
- Modelo de Prototipos: se basa en versiones preliminares del sistema antes del desarrollo final.

2.1.1.4 Cuadro comparativo

Modelo	Enfoque	Ventajas	Desventajas
Cascada	Secuencial	Fase clara, fácil de entender	Poco flexible, difícil de adaptarse a cambios
V	Iterativo	Buen control de pruebas	Puede ser costoso en tiempo y recursos
Espiral	Iterativo	Control de riesgos, flexibilidad	Puede ser complejo y costoso
Prototipos	Iterativo	Feedback temprano del usuario	Riesgo de expectativas no cumplidas

Incremental (usado)	Iterativo	Flexibilidad, retroalimentación continua, módulos independientes	Puede ser difícil gestionar incrementos grandes si no se planifican bien
--------------------------------	-----------	---	--

2.1.2 Metodología a usar: Metodología Incremental

La metodología incremental combina las ventajas del enfoque secuencial con las iteraciones progresivas. En este modelo, el desarrollo del sistema se divide en módulos o incrementos que se construyen y entregan de forma parcial, pero funcional.

Cada incremento añade características nuevas al sistema, permitiendo obtener retroalimentación continua.

Según Wrike (2024):

“Un enfoque incremental divide el proceso de desarrollo de software en pequeñas porciones manejables conocidas como incrementos. Cada incremento se basa en la versión anterior, de modo que las mejoras se realizan paso a paso.” (Wrike, What is Iterative Incremental Development).

Características

- Desarrollo por etapas o módulos.
- Permite entregas funcionales tempranas.
- Facilita la retroalimentación y corrección de errores.
- Flexibilidad ante cambios de requerimientos.
- Mejora continua del sistema.

Fases del enfoque incremental:

- **Planificación:** Se definen los módulos o incrementos a desarrollar, sus características y prioridades.
- **Desarrollo:** El sistema se divide en módulos independientes que se desarrollan y entregan de manera incremental.
- **Pruebas:** Cada módulo se prueba de manera independiente y en conjunto con otros módulos desarrollados previamente.
- **Entrega:** Se entrega un producto funcional con cada incremento, permitiendo su uso parcial mientras se siguen agregando más módulos.

Se eligió esta metodología porque se adapta perfectamente al sistema modular propuesto para la gestión de residencias profesionales. Al dividir el sistema en submódulos (registro de proyectos, asignación de asesores, evaluación, etc.), se pueden desarrollar e implementar de forma independiente y validar cada módulo antes de integrarlo al sistema completo. Esto reduce riesgos y optimiza los tiempos de entrega.

2.2 Frameworks

2.2.1 Front-end

2.2.1.1 Definición

El front-end o capa de presentación es la parte visible del sistema que interactúa directamente con el usuario. Incluye todos los elementos visuales e interfaces gráficas que permiten la comunicación entre el usuario y la aplicación.

De acuerdo con Mozilla (2023),

“El desarrollo front-end se refiere a la parte del desarrollo web que crea la interfaz de usuario: la parte del sitio web o la aplicación que los usuarios ven e interactúan.”

2.2.1.2 Características

- Se ejecuta en el navegador del usuario.
- Utiliza tecnologías como HTML, CSS y JavaScript.
- Se apoya en frameworks modernos para crear interfaces dinámicas (Angular, React, Vue.js).
- Enfocado en la experiencia de usuario (UX) y usabilidad (UI).
- Permite consumo de APIs del servidor (back-end).

2.2.1.3 Cuadro comparativo

Framework	Lenguaje Base	Ventajas	Desventajas
Angular (usado)	TypeScript	Modular, robusto, escalable, herramientas de Google integradas	Curva de aprendizaje pronunciada
React	JavaScript	Flexible, gran ecosistema, ligero	Necesita muchas librerías adicionales para características completas
Vue.js	JavaScript	Ligero, fácil de aprender, flexible	Ecosistema más pequeño que React

2.2.1.4 Framework de desarrollo web a utilizar: Angular

Angular es un framework de desarrollo de aplicaciones web mantenido por Google. Está basado en TypeScript y diseñado para construir aplicaciones de una sola página (SPA) con componentes reutilizables.

Según su documentación oficial:

“Angular es un framework web que permite a los desarrolladores crear aplicaciones rápidas y fiables. Mantenido por Google, proporciona herramientas, API y bibliotecas para simplificar y agilizar el desarrollo.” (Angular.dev, 2024).

La elección de Angular para el front-end se debe a su robustez, modularidad y compatibilidad con proyectos empresariales. Gracias a su arquitectura basada en componentes, facilita el desarrollo de interfaces escalables y mantenibles, ideales para gestionar los diferentes módulos del sistema de residencias profesionales. Además, su integración con APIs y herramientas reactivas mejora la experiencia del usuario y el rendimiento del sistema.

2.2.2 Back-end

2.2.2.1 Definición

El back-end o capa de servidor gestiona la lógica de negocio, la conexión con las bases de datos y las operaciones que el usuario no ve directamente.

Microsoft (2024) define:

“ASP.NET is an open-source web framework created by Microsoft for building modern web apps and services that run on Windows, Linux, macOS, and Docker.” (Microsoft Learn, 2024).

2.2.2.2 Características

- Maneja la lógica de negocio, seguridad y acceso a datos.
- Se ejecuta en el servidor y responde a peticiones HTTP del cliente.
- Permite el desarrollo de APIs RESTful para comunicación con el front-end.
- Soporta autenticación, validación y control de sesiones.
- Compatible con diversos gestores de base de datos.

2.2.2.3 Cuadro comparativo

Framework	Lenguaje	Ventajas	Desventajas
ASP.NET Core (usado)	C#	Alto rendimiento, fácil integración con bases de datos, multiplataforma	Curva de aprendizaje alta
Node.js	JavaScript	Asincrónico, eficiente, gran comunidad	No es ideal para operaciones muy intensivas en CPU
Django	Python	Rápido, muchas herramientas integradas	No tan flexible como otros

2.2.2.4 Framework de desarrollo de servicios web a utilizar: ASP.NET

ASP.NET Core es un framework open-source de Microsoft que permite crear aplicaciones web de alto rendimiento. Su arquitectura MVC facilita la separación de las capas de la aplicación y permite que el sistema sea escalable y mantenible. ASP.NET Core es compatible con MySQL y otros motores de bases de datos, lo que asegura la integridad y estabilidad del sistema.

Razón para elegir ASP.NET Core:

- Rendimiento y escalabilidad: ASP.NET Core es conocido por su alta eficiencia y capacidad de manejar grandes volúmenes de tráfico.
- Arquitectura MVC: Facilita el mantenimiento y la extensión del sistema.
- Compatibilidad multiplataforma: Funciona en Windows, Linux y macOS, lo que hace que el desarrollo sea más flexible.

2.3 Sistema gestor de Base de Datos

2.3.1 Definición

Un Sistema Gestor de Base de Datos (SGBD) es un software que permite crear, administrar y manipular bases de datos de manera estructurada.

Según Connolly y Begg (2015):

“A Database Management System (DBMS) is a software system that enables users to define, create, maintain, and control access to the database.”

2.3.2 Funciones

- Definir estructuras de datos y relaciones.
- Permitir operaciones CRUD (crear, leer, actualizar, eliminar).
- Garantizar la integridad y seguridad de los datos.
- Optimizar consultas mediante índices y procedimientos almacenados.
- Controlar el acceso concurrente de múltiples usuarios.

2.3.3 Cuadro comparativo de Sistemas Gestores de Base de Datos

SGBD	Tipo	Ventajas	Desventajas
MySQL (usado)	Relacional	Rápido, fiable, gratuito	Requiere mayor configuración para alta disponibilidad
PostgreSQL	Relacional	Flexible, compatible con tipos de datos complejos	Más lento que MySQL en ciertas operaciones
MongoDB	NoSQL	Flexible, ideal para grandes volúmenes de datos no estructurados	Menos adecuado para datos altamente relacionales

2.3.4 Sistema Gestor de Base de Datos a utilizar: MySQL

MySQL es un sistema gestor de bases de datos relacional ampliamente utilizado. Es gratuito, de código abierto y es conocido por su rendimiento y escalabilidad. MySQL se integra fácilmente con ASP.NET Core, y su naturaleza relacional es ideal para garantizar la integridad de los datos en sistemas de gestión como el de residencias profesionales.

Razón para elegir MySQL:

- Integración fácil con ASP.NET Core y su rendimiento probado.
- Gratuito y ampliamente adoptado, lo que facilita su implementación y mantenimiento.
- Alta disponibilidad y escalabilidad, lo que permite gestionar grandes volúmenes de datos institucionales.

CAPITULO III. DESARROLLO METODOLOGICO

3.1 Incremento 0: Análisis y Planeación del Proyecto

El incremento 0 constituye la etapa inicial de la metodología incremental. Su objetivo principal es establecer las bases conceptuales, técnicas y organizativas necesarias para el desarrollo del sistema web propuesto. En esta fase se definen los requerimientos generales, la arquitectura tecnológica, los recursos a utilizar, la planificación de entregas y la configuración del entorno de desarrollo.

El resultado de este incremento es un plan de trabajo estructurado y un entorno funcional para comenzar la implementación de los módulos posteriores.

3.1.1 Análisis de Requerimientos

Durante esta etapa se identificaron los requerimientos funcionales y no funcionales del sistema a partir de entrevistas con el personal de la Oficina de Proyectos de Vinculación, docentes y estudiantes.

Los requerimientos fueron organizados de la siguiente manera:

Requerimientos funcionales:

- Registro de usuarios con diferentes roles: estudiantes, docentes, asesores y administradores.
- Módulo para el registro y gestión de proyectos de residencia.
- Asignación de asesores y revisores a los proyectos.
- Control de entregas de reportes y documentos oficiales.
- Seguimiento del estado de cada residencia (en proceso, aprobado, concluido).
- Repositorio digital de proyectos concluidos y documentos validados.
- Panel de administración para la Oficina de Vinculación.
- Sistema de notificaciones y control de fechas clave.

Requerimientos no funcionales:

- Interfaz web accesible y responsive.
- Seguridad en la autenticación y manejo de información mediante encriptación.
- Escalabilidad para soportar múltiples usuarios concurrentes.
- Compatibilidad con navegadores modernos.
- Base de datos relacional para asegurar integridad y consistencia.
- Registro de actividad para trazabilidad y auditoría de acciones.

3.1.2 Análisis del entorno tecnológico

El entorno tecnológico fue seleccionado considerando la disponibilidad institucional, la compatibilidad con sistemas existentes y la facilidad de mantenimiento.

Herramientas principales:

- Front-end: Angular (TypeScript, HTML5, CSS3, Bootstrap).
- Back-end: ASP.NET Core 8 con arquitectura MVC.
- Base de datos: MySQL.
- Servidor de desarrollo: IIS Express y servidor de pruebas en Azure.
- Control de versiones: GitHub y GitHub Projects.
- Entorno de desarrollo: Visual Studio 2022 y Visual Studio Code.

Motivos de selección:

Angular proporciona una interfaz dinámica y modular ideal para aplicaciones institucionales, mientras que ASP.NET Core ofrece seguridad, rendimiento y escalabilidad. MySQL se eligió por su estabilidad, compatibilidad multiplataforma y soporte en entornos educativos.

3.1.3 Diseño general del sistema

En esta fase se estableció la arquitectura del sistema y los componentes iniciales.

Arquitectura general:

El sistema seguirá un modelo cliente-servidor de tres capas:

- Capa de presentación (Front-end): interfaz de usuario desarrollada en Angular.
- Capa de lógica de negocio (Back-end): servicios web RESTful implementados en ASP.NET Core.
- Capa de datos: base de datos MySQL que almacena la información de usuarios, proyectos y documentos.

Modelo Entidad-Relación preliminar:

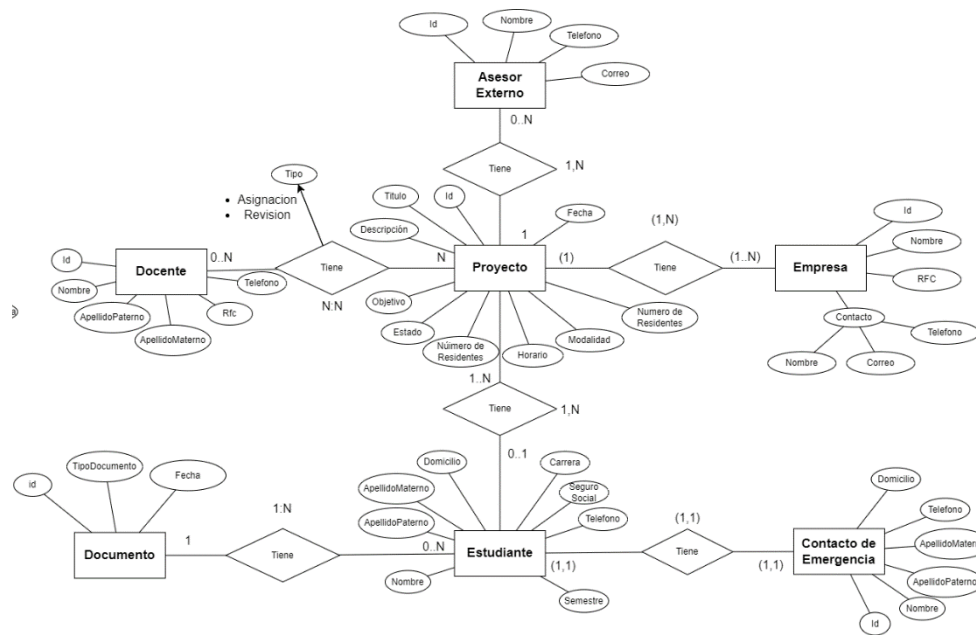
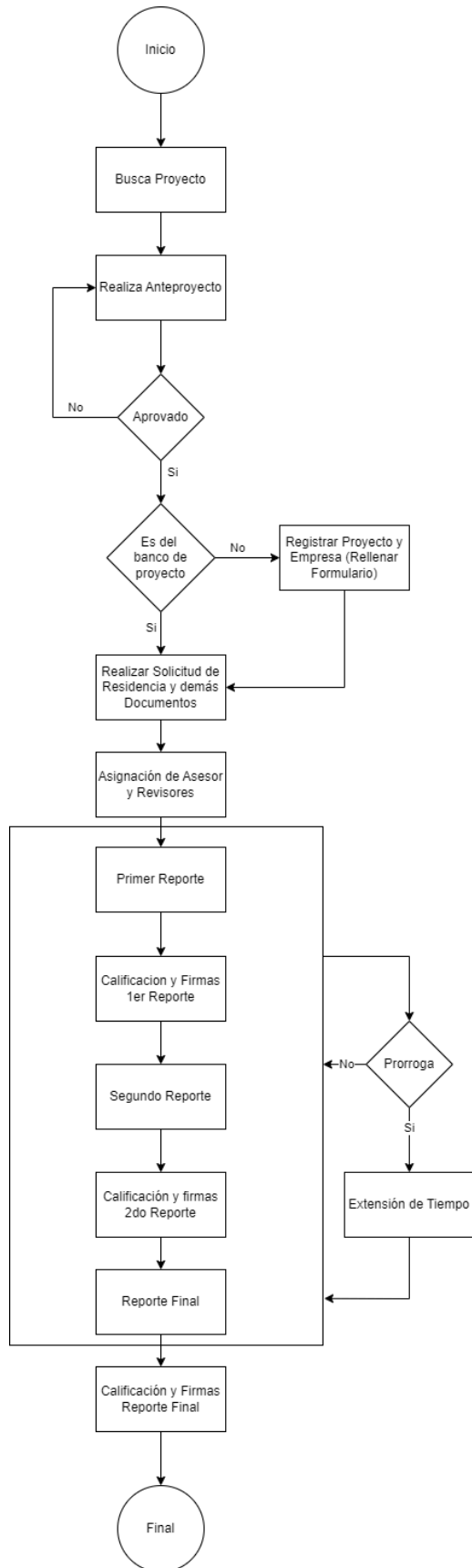


Diagrama de flujo Preliminar:



3.1.4 Planificación del Proyecto

El desarrollo se organizará en incrementos sucesivos, cada uno con entregables funcionales.

[illegible]

[illegible]

3.1.5 Configuración del entorno de desarrollo

En esta etapa se realizó la instalación, configuración y pruebas iniciales del entorno de trabajo:

- Instalación de Visual Studio 2022 con los paquetes de desarrollo para .NET 8.0.
- Instalación de Node.js y configuración del entorno Angular CLI.
- Creación del repositorio principal en GitHub y definición de ramas (main, develop, feature/).
- Instalación de MySQL Server y creación del esquema base.
- Configuración de entorno de pruebas con IIS Express y Azure App Service.
- Pruebas de conexión entre el front-end, back-end y base de datos.

Con esto, se garantizó la infraestructura técnica necesaria para iniciar el desarrollo del Incremento 1, asegurando la interoperabilidad entre las capas y un flujo de trabajo controlado.

3.1.6 Pruebas e Integración

3.1.6.1 Smoke tests (mínimos obligatorios)

- CP0-1 - Levantar frontend
 - Resultado esperado: carga sin errores.
- CP0-2 - Levantar backend
 - Resultado esperado: responde healthcheck o Swagger.
- CP0-3 - Conectividad frontend-backend
 - Resultado esperado: una llamada simple responde OK (por ejemplo, ping).
- CP0-4 - Conexión a BD (si aplica)
 - Resultado esperado: el backend puede consultar/crear un registro de prueba.

3.1.6.2 Incidencias y correcciones

Registro de incidencias detectadas durante la configuración inicial (completar según aplique):

Incidencia	Causa	Solución	Estado
(Ej. Error al iniciar la API)	(Ej. Cadena de conexión incorrecta)	(Ej. Ajuste en appsettings/env)	Abierta/Cerrada

3.1.7 Entrega y Retroalimentación

3.1.7.1 Entregables del Incremento 0 (checklist)

- Repositorio creado y organizado (estructura base).
- README con pasos para correr (frontend y backend).
- Variables de entorno documentadas.
- Aplicación frontend ejecuta localmente.
- API backend ejecuta localmente.
- Conectividad frontend-backend validada.
- (Si aplica) Conexión a base de datos validada.
- (Si aplica) Autenticación base funcionando.

3.1.7.2 Retroalimentación recibida (si hubo)

Comentarios:

Ajustes:

3.1.7.3 Pendientes para Incremento 1

- Pendiente 1: (completar)
- Pendiente 2: (completar)

3.1.7.4 Lecciones aprendidas

(Escribir 3-5 líneas sobre lo aprendido en esta fase inicial.)

3.1.7.5 Retrospectiva breve

Qué salió bien:

Qué se complicó:

Qué harías distinto:

3.2 Incremento 1: Login y Módulos de registro (estudiantes, docentes, proyectos y empresas).

Entregable: Módulos de Login + Registro funcionando en ambiente local y evidencias de pruebas.

- Asignación de asesores/revisores y generación de documentos (incrementos posteriores).
- Banco de proyectos y flujo de anteproyecto (se documenta en el Incremento 2).

Fuera de alcance (no incluye):

- Evidencia de funcionamiento: capturas de UI y pruebas de API.
- Validaciones de campos obligatorios y consistencia de relaciones (por ejemplo, proyecto asociado a empresa).
- Pantallas y endpoints para alta, edición, consulta y baja (si aplica) de Estudiantes, Docentes, Empresas y Proyectos.
- Autenticación y autorización (roles/permisos) operativa.

Alcance (incluye):

Objetivo: Implementar el inicio de sesión con control de acceso por roles y los módulos de registro (CRUD) de Estudiantes, Docentes, Empresas y Proyectos, asegurando validaciones básicas y trazabilidad de los datos.

3.2.1 Planificación y Análisis de Requisitos

En este incremento se definieron los requisitos para el acceso al sistema y el registro de entidades principales. Se priorizó asegurar la integridad de datos y el control de acceso por roles.

Requisitos funcionales (resumen):

- RF-01 Iniciar sesión con credenciales válidas.
- RF-02 Restringir pantallas/acciones según rol (autorización).
- RF-03 Registrar, editar y consultar estudiantes.
- RF-04 Registrar, editar y consultar docentes.
- RF-05 Registrar, editar y consultar empresas.
- RF-06 Registrar, editar y consultar proyectos.
- RF-07 Validar datos obligatorios y evitar duplicados clave (por ejemplo, correo/no. control) según aplique.

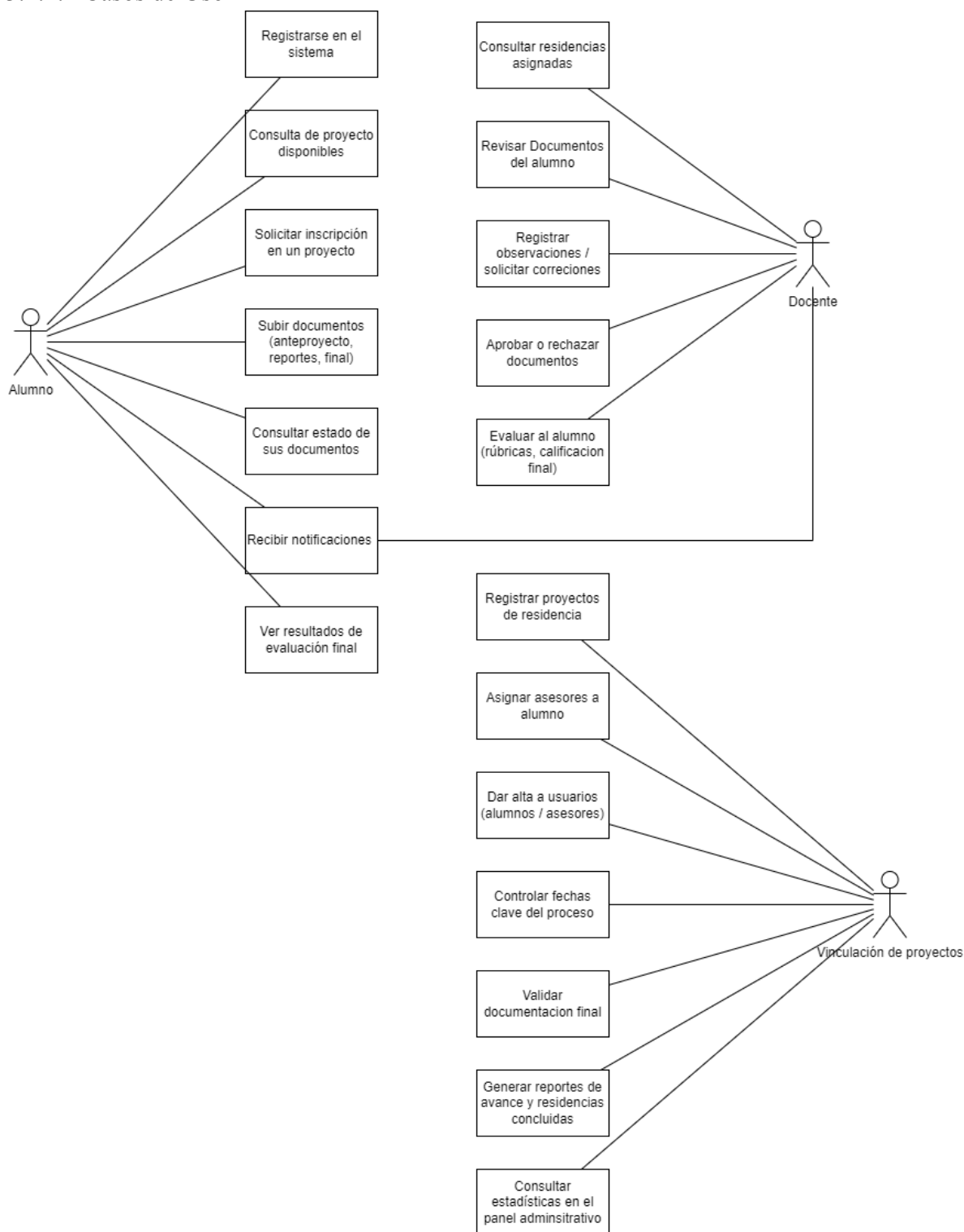
Reglas de negocio (mínimas) consideradas:

- RB-01 Un usuario debe autenticarse para acceder a módulos internos.
- RB-02 Las acciones de alta/edición/baja se controlan por rol/permisos.
- RB-03 Los campos obligatorios no pueden enviarse vacíos.
- RB-04 Un proyecto debe estar asociado a una empresa registrada.
- RB-05 Identificadores institucionales (correo/no. control/no. empleado) deben ser únicos si el sistema los usa como llave.

Matriz de roles y permisos (completar según la implementación):

Rol	Login	Reg. Estudiantes	Reg. Docentes	Reg. Proyectos/Empr esas
Administrador/ Oficina	Sí	Alta/Edición/C onсульта	Alta/Edición/C onсульта	Alta/Edición/C onсульта
Docente	Sí	Consulta	Consulta	Consulta
Estudiante	Sí	Consulta propia	N/A	Consulta
Revisor (si aplica)	Sí	Consulta	Consulta	Consulta

3.2.1.1 Casos de Uso



A continuación se listan los casos de uso principales del incremento. Los casos detallados pueden ampliarse si el asesor lo requiere.

Listado de casos de uso:

ID	Nombre	Actor principal	Resultado
UC-01	Iniciar sesión	Usuario	Acceso concedido según rol
UC-02	Registrar estudiante	Administrador/Oficina	Estudiante registrado
UC-03	Registrar docente	Administrador/Oficina	Docente registrado
UC-04	Registrar empresa	Administrador/Oficina	Empresa registrada
UC-05	Registrar proyecto	Administrador/Oficina	Proyecto registrado y ligado a empresa
UC-06	Consultar listados	Usuario autorizado	Listado/consulta mostrada

Especificación resumida (UC-01 Iniciar sesión):

- Actor: Usuario.
- Precondiciones: Usuario registrado y activo.
- Flujo principal: (1) Ingresa usuario y contraseña. (2) El sistema valida credenciales. (3) Se emite sesión/token. (4) Se redirige a la vista permitida por rol.
- Flujos alternos: Credenciales inválidas; usuario inactivo; falta de permisos.
- Postcondiciones: Sesión iniciada y permisos cargados.

Especificación resumida (UC-05 Registrar proyecto):

- Actor: Administrador/Oficina.
- Precondiciones: Empresa registrada; usuario con permiso de alta de proyectos.
- Flujo principal: (1) Captura datos del proyecto. (2) Selecciona empresa asociada. (3) El sistema valida campos. (4) Se guarda el proyecto con su relación.
- Flujos alternos: Empresa no existe; campos obligatorios vacíos; valores duplicados si aplica.
- Postcondiciones: Proyecto registrado y visible en listados.

3.2.1.2 Historias de Usuario

Historia de usuario	
Número: 1	Usuario: Administrador
Nombre historia:	Inicio de sesión
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como usuario, quiero ingresar al sistema mediante un login para acceder a las funcionalidades de gestión.
Validación:	Se permite el acceso con credenciales válidas; credenciales inválidas muestran mensaje de error y no inician sesión.
Historia de usuario	

Número: 2	Usuario: Administrador
Nombre historia:	Cierre de sesión
Prioridad en empresa: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 0.5 días	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como usuario, quiero poder cerrar mi sesión para proteger el acceso al sistema.
Validación:	Un botón “Cerrar sesión” invalida el token/estado y redirige a la pantalla de login.
Historia de usuario	
Número: 3	Usuario: Administrador
Nombre historia:	Cambio contraseña
Prioridad en empresa: Media	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como usuario, quiero restablecer mi contraseña si la olvido para recuperar el acceso.
Validación:	Se envía correo con contraseña temporal/enlace; tras restablecer, puede iniciar sesión con la nueva credencial.
Historia de usuario	
Número: 4	Usuario: Administrador
Nombre historia:	Registrar estudiante
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero registrar estudiantes individualmente para habilitarlos en el sistema.
Validación:	Guarda alumno con nombre, apellidos, correo institucional y no. de control; crea/relaciona usuario y muestra confirmación.
Historia de usuario	
Número: 5	Usuario: Administrador
Nombre historia:	Editar estudiante
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 2
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero actualizar la información de un estudiante para mantener datos vigentes.

Validación:	Al seleccionar un estudiante, el formulario se llena y permite actualizar; se muestra mensaje de éxito.
Historia de usuario	
Número: 6	Usuario: Administrador
Nombre historia:	Carga masiva de estudiantes (Excel)
Prioridad en empresa: Alta	Riesgo en desarrollo: Alto
Puntos estimados: 2 días	Iteración asignada: 2
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero cargar estudiantes desde Excel para agilizar el registro.
Validación:	Lee encabezados mínimos; valida filas; salta duplicados (correo/no. control) y continúa; barra de progreso y resumen por fila.
Historia de usuario	
Número: 8	Usuario: Administrador
Nombre historia:	Registrar docente
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero registrar docentes para habilitarlos como tutores/asesores.
Validación:	Guarda docente con nombre, apellidos y correo; crea/relaciona usuario; muestra confirmación.
Historia de usuario	
Número: 9	Usuario: Administrador
Nombre historia:	Editar docente
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 2
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero actualizar la información de un docente.
Validación:	Al editar, los campos se precargan; se guardan cambios y se notifica éxito.
Historia de usuario	
Número: 10	Usuario: Administrador
Nombre historia:	Carga masiva de docentes (Excel)
Prioridad en empresa: Media	Riesgo en desarrollo: Alto
Puntos estimados: 2 días	Iteración asignada: 2
Programador responsable: Hernández Ramírez Osiris	

Descripción:	Yo, como administrador, quiero cargar docentes desde Excel para agilizar el registro.
Validación:	Valida encabezados; salta correos duplicados y continúa; barra de progreso y reporte por fila.
Historia de usuario	
Número: 12	Usuario: Administrador
Nombre historia:	Registrar empresa
Prioridad en empresa: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 1 día	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero registrar empresas para vincular proyectos.
Validación:	Guarda nombre obligatorio; opcional RFC/teléfono/correo; muestra confirmación y actualiza la lista.
Historia de usuario	
Número: 13	Usuario: Administrador
Nombre historia:	Editar empresa
Prioridad en empresa: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 1 día	Iteración asignada: 2
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero actualizar datos de una empresa.
Validación:	Precarga datos en formulario; guarda cambios y confirma éxito.
Historia de usuario	
Número: 15	Usuario: Administrador
Nombre historia:	Registrar proyecto (FK empresa)
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1.5 días	Iteración asignada: 1
Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero registrar proyectos vinculados a una empresa para gestionar residencias.
Validación:	Formulario requiere selección de empresa existente, título y noResidentes $\geq$ 1; guarda horario inicio/fin y marca de Home Office.
Historia de usuario	
Número: 16	Usuario: Administrador
Nombre historia:	Editar proyecto
Prioridad en empresa: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1 día	Iteración asignada: 2

Programador responsable: Hernández Ramírez Osiris	
Descripción:	Yo, como administrador, quiero actualizar la información de un proyecto.
Validación:	Precarga datos; permite cambio de empresa, estado, objetivo, horario y noResidentes; guarda y confirma.

Las historias de usuario del incremento se enfocan en permitir el acceso controlado por roles y la gestión de registros base. Se muestran con criterios de aceptación verificables.

Resumen de historias de usuario:

HU-ID	Como (rol)	Quiero	Para	Criterios de aceptación (resumen)
HU-01	Usuario	iniciar sesión	acceder al sistema	Con credenciales válidas se permite acceso; con inválidas se muestra mensaje y no se inicia sesión.
HU-02	Administrador/ Oficina	gestionar estudiantes (alta/edición/consulta)	mantener el padrón actualizado	Campos obligatorios validados; registro se refleja en listado; se impide duplicado clave si aplica.
HU-03	Administrador/ Oficina	gestionar docentes	administrar participantes del sistema	Validación de correo institucional; roles/permisos asignables según reglas.
HU-04	Administrador/ Oficina	gestionar empresas	registrar entidades receptoras	RFC/campos obligatorios validados según aplique; empresa visible para asociar proyectos.
HU-05	Administrador/ Oficina	gestionar proyectos	vincular estudiantes y empresa	No permite registrar proyecto sin

				empresa; el estado inicial se asigna conforme a reglas.
HU-06	Administrador	controlar permisos por rol	restringir accesos	Usuarios sin permiso no pueden ver/ejecutar acciones; acciones autorizadas funcionan.

Notas:

- Ajustar roles reales del sistema y permisos específicos conforme a lo implementado.
- Si existe restablecimiento de contraseña por correo, anexar evidencia (si no existe, documentar el método real).

3.2.2 Diseño y Desarrollo

En esta fase se implementó el flujo de autenticación/autorización y los módulos de registro. Se mantuvo separación por capas (presentación, servicios/API y persistencia) para facilitar mantenibilidad y pruebas.

Arquitectura (resumen):

- Frontend: vistas de Login y pantallas CRUD para Estudiantes, Docentes, Empresas y Proyectos.
- Backend/API: endpoints REST para autenticación, autorización y operaciones CRUD.
- Base de datos: tablas para usuarios/roles y entidades del dominio, con llaves foráneas para integridad.

Endpoints implementados (completar con rutas reales):

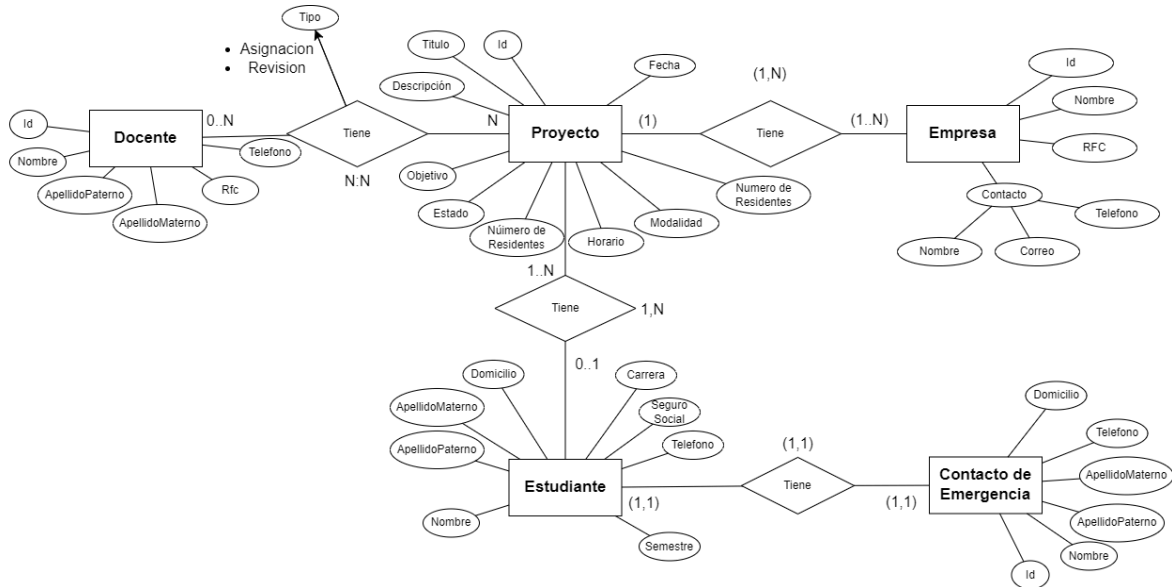
Módulo	Endpoint	Método	Descripción	Protección (rol/permisos)
Auth	/api/auth/login	POST	Inicia sesión y devuelve token/sesión	Público
Estudiantes	/api/students	GET/POST	Lista y registra estudiantes	Admin/Oficina
Docentes	/api/teachers	GET/POST	Lista y registra docentes	Admin/Oficina
Empresas	/api/companies	GET/POST	Lista y registra empresas	Admin/Oficina
Proyectos	/api/projects	GET/POST	Lista y registra proyectos	Admin/Oficina

Catálogos (si aplica)	/api/catalogs/*	GET	Estados, periodos, etc.	Autenticado
-----------------------	-----------------	-----	-------------------------	-------------

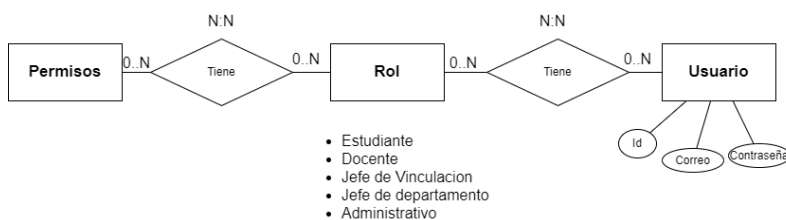
Evidencia recomendada a anexar:

- Captura de pantalla: Login exitoso con un rol.
- Captura de pantalla: listado y formulario de alta (al menos un módulo).
- Captura de Swagger/Postman: respuesta de login y una operación CRUD.
- Captura de base de datos: registro creado (si aplica).

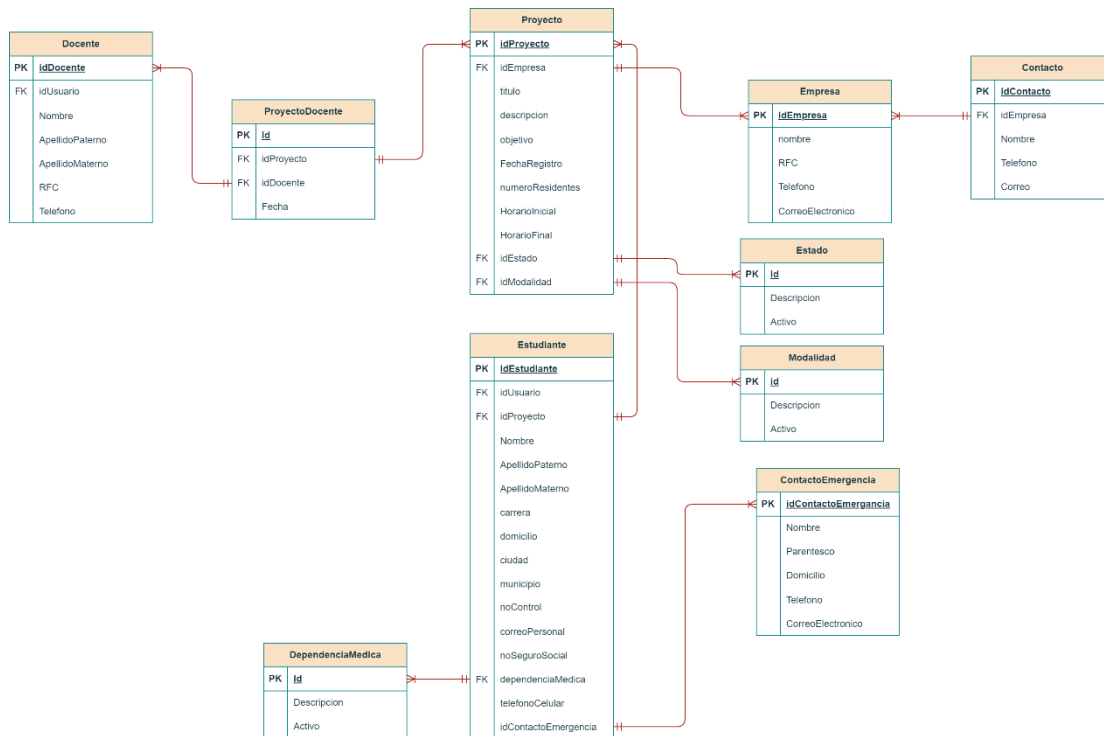
3.2.2.1 Diagrama Entidad-Relación



3.2.2.3 Diagrama Entidad-Relación para la funcionalidad del sistema



3.2.2.4 Diagrama Relacional



3.2.2.5 Diagrama Relacional para el funcionamiento del sistema



3.2.2.6 Diccionario de Datos

Usuario

Campo	Descripción	Tipo	Long.	P K	FK (tabla, campo)	Nul o	Reglas / Descripción	Índice Único
idUsuario	Identificador único del	INT		Sí		No	Identificador auto	PK

	usuario; se genera automáticamente por el sistema.						incremental.	
correoInstitucional	Correo electrónico institucional del usuario.	VARCHAR	150	No		No	Debe tener formato de correo electrónico válido. No se permiten duplicados.	Único
Password	Contraseña de acceso al sistema asociada al usuario.	VARCHAR	150	No		No	Almacenar de forma cifrada. Longitud mínima sugerida: 8 caracteres.	
Activo	Indica si el usuario tiene acceso activo al sistema.	BIT		No		No	1 = Activo, 0 = Inactivo.	

Rol

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción	Índice Único
Id	Identificador único del rol.	INT		Sí		No	Identificador autoincremental.	PK
Descripción	Nombre o descripción del rol del usuario.	VARCHAR	100	No		No	Valores típicos: Estudiante, Docente, Jefe de	

							Vinculación, etc.	
Activo	Indica si el rol puede asignarse a nuevos usuarios.	BIT		No		No	1 = Activo, 0 = Inactivo.	

Permisos

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción /	Índice Único
Id	Identificador único del permiso.	INT		Sí		No	Identificador autoincremental.	PK
Descripción	Descripción del permiso o acción que se puede realizar en el sistema.	VARCHAR	150	No		No	Ejemplos: CrearProyecto, AprobarProyecto, RegistrarEstudiante.	
Activo	Indica si el permiso está disponible para ser asignado a roles.	BIT		No		No	1 = Activo, 0 = Inactivo.	

UsuarioRol

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción /	Índice Único
Id	Identificador único del registro de	INT		Sí		No	Identificador autoincremental.	PK

	relación usuario-rol.							
IdUsuario	Referencia al usuario al que se le asigna el rol.	INT		No	Usuario.idUsuario	No	Debe existir previamente en la tabla Usuario.	
IdRol	Referencia al rol asignado al usuario.	INT		No	Rol.Id	No	Debe existir previamente en la tabla Rol.	

RolPermiso

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción	Índice Único
Id	Identificador único del registro de relación rol-permiso.	INT		Sí		No	Identificador autoincremental.	PK
Id_Rol	Rol al que se le asigna el permiso.	INT		No	Rol.Id	No	Debe existir en la tabla Rol.	
Id_Permiso	Permiso asignado al rol.	INT		No	Permisos.Id	No	Debe existir en la tabla Permisos.	

Empresa

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción	Índice Único
idEmpresa	Identificador único de la empresa receptora de residentes.	INT		Sí		No	Identificador autoincremental.	PK

nombre	Nombre comercial o razón social de la empresa.	VARCHAR	150	No		No		
RFC	Registro Federal de Contribuyentes de la empresa.	VARCHAR	13	No		No	Debe cumplir con el formato oficial de RFC en México.	Único
Telefono	Número telefónico principal de contacto de la empresa.	VARCHAR	20	No		Sí		
CorreoElectronico	Correo electrónico general de la empresa.	VARCHAR	150	No		Sí	Formato de correo válido.	

ContactoEmpresa

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas / Descripción	Índice Único
idContacto	Identificador único del contacto de la empresa.	INT		Sí		No	Identificador autoincremental.	PK
idEmpresa	Referencia a la empresa a la que pertenece el contacto.	INT		No	Empresa.idEmpresa	No	Debe existir en la tabla Empresa.	
Nombre	Nombre completo de la persona de contacto.	VARCHAR	150	No		No		

Telefono	Teléfono directo del contacto.	VARCHAR	20	No		Sí		
Correo	Correo electrónico del contacto.	VARCHAR	150	No		Sí	Formato de correo válido.	

EstadoProyecto

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción /	Índice Único
Id	Identificador del estado del proyecto.	INT		Sí		No	Identificador autoincremental.	PK
Descripción	Descripción del estado del proyecto.	VARCHAR	100	No		No	Ejemplos: Registrado, En revisión, Aprobado, Rechazado.	
Activo	Indica si el estado está vigente para usarse en nuevos proyectos.	BIT		No		No	1 = Activo, 0 = Inactivo.	

Modalidad

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción /	Índice Único
Id	Identificador de la modalidad del proyecto.	INT		Sí		No	Identificador autoincremental.	PK
Descripción	Descripción de la	VARCHAR	100	No		No	Ejemplos: Presencial,	

	modalidad						Virtual, Mixta.	
Activo	Indica si la modalidad está vigente.	BIT		No		No	1 = Activo, 0 = Inactivo.	

Proyecto

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nu lo	Reglas / Descripción	Índice Único
idProyecto	Identificador único del proyecto de residencia.	INT		Sí		No	Identificador autoincremental.	PK
idEmpresa	Referencia a la empresa donde se desarrolla el proyecto.	INT		No	Empresa.idEmpresa	No	Debe existir en la tabla Empresa.	
titulo	Título del proyecto.	VARCHAR	200	No		No		
descripcion	Descripción general del proyecto.	TEXT		No		Sí		
objetivo	Objetivo principal del proyecto.	TEXT		No		Sí		
FechaRegistro	Fecha en que se	DATE		No		No		

	registró el proyecto .							
numeroResidentes	Número de estudiantes residentes asignados al proyecto .	INT		No		No	Debe ser un valor mayor o igual a 0.	
HorarioInicial	Hora de inicio de la jornada de residencia.	TIME		No		Sí		
HorarioFinal	Hora de término de la jornada de residencia.	TIME		No		Sí		
idEstado	Estado actual del proyecto .	INT		No	EstadoProyecto.Id	No	Debe existir en la tabla EstadoProyecto.	
idModalidad	Modalidad de ejecución del proyecto .	INT		No	Modalidad.Id	No	Debe existir en la tabla Modalidad.	

Docente

Campo	Descripción	Tipo	Long.	P K	FK (tabla, campo)	Nulo	Reglas / Descripción	Índice Único
-------	-------------	------	-------	-----	-------------------	------	----------------------	--------------

idDocente	Identificador único del docente.	INT		Sí		No	Identificador autoincremental.	PK
idUsuario	Referencia al usuario asociado al docente.	INT		No	Usuario.idUsuario	No	Debe existir en la tabla Usuario.	
Nombre	Nombre del docente.	VARCHAR	100	No		No		
ApellidoPaterno	Apellido paterno del docente.	VARCHAR	100	No		No		
ApellidoMaterno	Apellido materno del docente.	VARCHAR	100	No		Sí		
RFC	Registro Federal de Contribuyentes del docente.	VARCHAR	13	No		Sí		
Telefono	Teléfono de contacto del docente.	VARCHAR	20	No		Sí		

TipoRelacion

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas Descripción /	Índice Único
Id	Identificador del tipo de relación docente-proyecto.	INT		Sí		No	Identificador autoincremental.	PK
Descripción	Tipo de relación, por	VARCHAR	100	No		No		

	ejemplo Asignación o Revisión.							
Activo	Indica si el tipo de relación está vigente.	BIT		No		No	1 = Activo, 0 = Inactivo.	

ProyectoDocente

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas / Descripción	Índice Único
Id	Identificador único del registro proyecto-docente.	INT		Sí		No	Identificador autoincremental.	PK
idProyecto	Proyecto asociado al docente.	INT		No	Proyecto.idProyecto	No	Debe existir en la tabla Proyecto.	
idDocente	Docente que participa en el proyecto.	INT		No	Docente.idDocente	No	Debe existir en la tabla Docente.	
TipoRelacion	Tipo de participación del docente.	INT		No	TipoRelacion.Id	No	Valores definidos en la tabla TipoRelacion.	
Fecha	Fecha de registro de la relación.	DATE		No		No		
Comentario	Comentario adicional sobre la relación.	TEXT		No		Sí		

Aprobado	Indica si el docente ha aprobado el proyecto.	BIT		No		No	1 = Aprobado, 0 = No aprobado.	
-----------------	---	-----	--	----	--	----	--------------------------------	--

Estudiante

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas / Descripción	Índice Único
idEstudiante	Identificador único del estudiante.	INT		Sí		No	Identificador autoincremental.	PK
idUsuario	Usuario asociado al estudiante.	INT		No	Usuario.idUsuario	No	Permite el inicio de sesión del estudiante.	
idProyecto	Proyecto en el que participa el estudiante.	INT		No	Proyecto.idProyecto	Sí	Puede ser nulo mientras el estudiante no tenga proyecto asignado.	
Nombre	Nombre del estudiante.	VARCHAR	100	No		No		
ApellidoPaterno	Apellido paterno del estudiante.	VARCHAR	100	No		No		
ApellidoMaterno	Apellido materno del	VARCHAR	100	No		Sí		

	estudiante.							
carrera	Programa educativo o carrera del estudiante.	VAR CHAR	10 0	N o		N o		
domicilio	Domicilio completo del estudiante.	VAR CHAR	20 0	N o		Sí		
ciudad	Ciudad de residencia del estudiante.	VAR CHAR	10 0	N o		Sí		
municipio	Municipio de residencia del estudiante.	VAR CHAR	10 0	N o		Sí		
noControl	Número de control del estudiante.	VAR CHAR	20	N o		N o	Debe ser único en la institución.	Único
correoPersonal	Correo personal del estudiante.	VAR CHAR	15 0	N o		Sí	Formato de correo válido.	
noSeguroSocial	Número de seguridad	VAR CHAR	20	N o		Sí		

	social del estudiante.							
dependenciaMedica	Dependencia médica a la que está afiliado.	VARCHAR	100	No		Sí		
telefonoCelular	Teléfono celular del estudiante.	VARCHAR	20	No		Sí		
idContactoEmergencia	Referencia al contacto de emergencia del estudiante.	INT		No	ContactoEmergencia.idContactoEmergencia	Sí	Debe existir en la tabla ContactoEmergencia.	

ContactoEmergencia

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas / Descripción	Índice Único
idContactoEmergencia	Identificador único del contacto de emergencia.	INT		Sí		No	Identificador autoincremental.	PK
Nombre	Nombre completo del contacto	VARCHAR	150	No		No		

	de emergencia.							
Parentesco	Relación del contacto con el estudiante.	VARCHAR	100	No		No	Ejemplos: Madre, Padre, Tutor.	
Domicilio	Domicilio del contacto de emergencia.	VARCHAR	200	No		Sí		
Telefono	Teléfono del contacto de emergencia.	VARCHAR	20	No		No		
CorreoElectronico	Correo electrónico del contacto de emergencia.	VARCHAR	150	No		Sí	Formato de correo válido.	

Documento

Campo	Descripción	Tipo	Long.	PK	FK (tabla, campo)	Nulo	Reglas / Descripción	Índice Único
idDocumento	Identificador único del documento asociado al estudiante.	INT		Sí		No	Identificado o autoincremental.	PK
idEstudiante	Estudiante al que	INT		No	Estudiante.idEstudiante	No	Debe existir en la	

	pertenec e el documen to.						tabla Estudiante.	
Tipo	Tipo de documen to.	VARCH AR	100	N o		No	Ejemplos: Carta de presentació n, Carta de aceptación, Reporte final.	
Fecha	Fecha de emisión o recepció n del documen to.	DATE		N o		No		

3.2.3 Pruebas e Integración

Se realizaron pruebas de integración entre frontend, backend y base de datos, así como verificación de permisos por rol. A continuación se presentan los casos mínimos recomendados (ajustar según implementación).

3.2.3.1 Login

Casos de prueba (CP1 - Login):

ID	Entrada	Resultado esperado	Evidencia
CP1-Login-01	Credenciales válidas	Acceso concedido y rol cargado	Captura UI + status 200
CP1-Login-02	Contraseña incorrecta	No inicia sesión; mensaje de error	Captura UI + status 401/400
CP1-Login-03	Usuario inexistente	No inicia sesión; mensaje controlado	Captura UI + status 404/401
CP1-Login-04	Usuario sin permiso (si aplica)	Acceso denegado a módulos restringidos	Captura permisos/403

3.2.3.2 Estudiantes

- CP1-Est-01 Alta con datos válidos -> registro creado y visible en listado.
- CP1-Est-02 Alta con campo obligatorio vacío -> error de validación.
- CP1-Est-03 Edición -> cambios persistidos.
- CP1-Est-04 Acceso por rol -> un rol sin permiso no puede crear/editar.

3.2.3.3 Docentes

- CP1-Doc-01 Alta con datos válidos -> registro creado.
- CP1-Doc-02 Validación de correo institucional/únicos (si aplica).
- CP1-Doc-03 Edición y consulta -> cambios visibles.
- CP1-Doc-04 Permisos por rol -> restricciones correctas.

3.2.3.4 Empresas

- CP1-Emp-01 Alta con datos válidos -> empresa creada.
- CP1-Emp-02 Validación de campos obligatorios (y RFC si aplica).
- CP1-Emp-03 Consulta/listado -> empresa disponible para asociar proyectos.

3.2.3.5 Proyectos

- CP1-Proy-01 Alta con datos válidos y empresa existente -> proyecto creado.
- CP1-Proy-02 Alta sin empresa -> validación impide guardar.
- CP1-Proy-03 Estado inicial asignado correctamente (según catálogo/regla).
- CP1-Proy-04 Permisos por rol -> solo roles autorizados pueden crear/editar.

Registro de incidencias detectadas durante pruebas (opcional):

Incidencia	Causa	Solución	Estado
(Completar)	(Completar)	(Completar)	Abierta/Cerrada

3.2.4 Entrega y Retroalimentación

3.2.4.1 Entregables del Incremento 1 (checklist)

- Login funcional con control de acceso por roles/permisos.
- Pantallas de registro (CRUD) implementadas para Estudiantes, Docentes, Empresas y Proyectos.
- Endpoints REST correspondientes documentados (Swagger/Postman).
- Validaciones mínimas implementadas (campos obligatorios, consistencia de relaciones).
- Evidencia de pruebas (capturas UI y respuestas API).

3.2.4.2 Retroalimentación recibida (si hubo)

Comentarios:

Ajustes:

3.2.4.3 Pendientes para Incremento 2

- Pendiente 1: (completar)
- Pendiente 2: (completar)

3.2.4.4 Lecciones aprendidas

(Escribir 3-5 líneas sobre lo aprendido al implementar login, permisos y CRUDs.)

3.3 Incremento 2: Banco de Proyectos y Anteproyecto

Fuera de alcance: Generación de documentos y asignación de revisores (incrementos 3+).

- Gestión de estados del proyecto relacionados al anteproyecto.
- Carga de archivo/registro del anteproyecto y su asociación al proyecto.
- Pantalla de banco de proyectos (búsqueda, filtros y detalle).

Alcance (incluye):

Objetivo: Habilitar el banco/listado de proyectos y la funcionalidad para cargar/gestionar el anteproyecto, incluyendo estados del proyecto asociados a este flujo.

3.3.1 Planificación y Análisis de Requisitos

- Criterios de aceptación (por ejemplo: validaciones de archivo, estados permitidos, roles autorizados).
- Casos de uso y/o historias de usuario (referenciar IDs).
- Requisitos funcionales (RF) y reglas de negocio del banco de proyectos y anteproyecto.

Contenido sugerido:

3.3.2 Diseño y Planificación

- Consideraciones de seguridad: permisos de carga/descarga/consulta por rol.
- Endpoints y contratos (DTOs) que soportan carga y consulta.
- Diseño de datos: tablas/relaciones nuevas (por ejemplo: Anteproyecto, Estados, Historial).
- Diseño de UI: pantallas, navegación, componentes (capturas o mockups).

Contenido sugerido:

3.3.3 Pruebas e Integración

- Incidencias detectadas y correcciones aplicadas (tabla).
- Evidencia (capturas UI y respuestas API).
- Matriz de pruebas (CP2-*) para: listado, filtros, carga de anteproyecto, validación de formatos, permisos por rol.

Contenido sugerido:

3.3.4 Entrega y Retroalimentación

- Lecciones aprendidas.
- Pendientes para el incremento 3.
- Retroalimentación recibida (comentarios y ajustes).
- Checklist de entregables del incremento 2.

Contenido sugerido:

3.4 Incremento 3: Asignación de Asesores y Revisores

Fuera de alcance: Expedientes completos y generación masiva de documentos (incrementos posteriores).

- Control de acceso por roles/permisos.
- Visualización de asignaciones en el detalle del proyecto.
- Asignación y desasignación de asesores/revisores.

Alcance (incluye):

Objetivo: Implementar la asignación de asesores y revisores a proyectos, incluyendo reglas de validación (etapas/estados) y visibilidad por rol.

3.4.1 Planificación y Análisis de Requisitos

- Consideraciones de auditoría: quién asignó, cuándo (trazabilidad).
- Casos de uso / historias de usuario con criterios de aceptación.
- Reglas de negocio: quién puede asignar, en qué etapas, límites (número de revisores, etc.).

Contenido sugerido:

3.4.2 Diseño y Planificación

- Seguridad: permisos por rol y respuestas 403 controladas.
- Endpoints: asignar/desasignar, listar revisores, etc. (completar rutas).
- Diseño de UI: diálogo de asignación, listas, validaciones visibles.
- Cambios en BD: tablas de relación (Proyecto-Docente, roles en proyecto, historial).

Contenido sugerido:

3.4.3 Pruebas e Integración

- Registro de incidencias y correcciones.
- Evidencia: capturas de UI y API; verificación en BD.
- Casos de prueba (CP3-*): asignación válida, inválida por etapa, sin permiso, duplicados, etc.

Contenido sugerido:

3.4.4 Entrega y Retroalimentación

- Lecciones aprendidas.
- Pendientes para incremento 4.
- Retroalimentación y ajustes.
- Checklist de entregables del incremento 3.

Contenido sugerido:

3.5 Incremento 4: Expedientes de Proyectos de Residencia

Fuera de alcance: Cierre administrativo final si se documenta en 3.6.

- Validaciones de completitud para avanzar etapas.
- Organización por tipo de documento y etapa.
- Carga/descarga/visualización de documentos del expediente.

Alcance (incluye):

Objetivo: Consolidar el expediente digital por proyecto (documentos, estados, evidencias), con control de versiones y acceso por rol.

3.5.1 Planificación y Análisis de Requisitos

- Reglas de seguridad (quién ve qué documentos).
- Casos de uso / historias de usuario.
- Lista de documentos del expediente y reglas de obligatoriedad por etapa.

Contenido sugerido:

3.5.2 Diseño y Planificación

- Endpoints para archivos: upload/download, validación de tipo y tamaño.
- Diseño de UI: visor/listado de documentos, carga con validaciones, estados.
- Modelo de datos para documentos (metadatos, almacenamiento, versión).

Contenido sugerido:

3.5.3 Pruebas e Integración

- Evidencia y registro de incidencias.
- Casos de prueba (CP4-*): carga válida, formato no permitido, sin permiso, descarga, etc.

Contenido sugerido:

3.5.4 Entrega y Retroalimentación

- Lecciones aprendidas.
- Pendientes finales/cierre.
- Retroalimentación y ajustes.
- Checklist de entregables del incremento 4.

Contenido sugerido:

3.6 Cierre del Proyecto

Referencias

- Sommerville, I. (2011). Software engineering (9th ed.). Addison-Wesley.
- Wrike. (2024). What is iterative incremental development. Wrike. <https://www.wrike.com/blog/iterative-incremental-development/>
- Mozilla. (2023). Front-end web development. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Learn/Front-end_web_developer
- Angular. (2024). Angular: The modern web framework. <https://angular.dev>
- Microsoft. (2024). What is ASP.NET Core? Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core>
- Connolly, T., & Begg, C. (2015). Database systems: A practical approach to design, implementation, and management (6th ed.). Pearson.
- MySQL. (2024). MySQL: The world's most popular open source database. <https://www.mysql.com>